



Department of Computer Science

Binary Search

Computational Core

Binary Search

0101010101010101
0101010101010101
0101010101010101
0101010101010101
0101010101010101



[This Photo](#) by Unknown Author
is licensed under [CC BY-SA](#)

Binary Search

1. Select the value in the middle of the container
 - a) If the value in the middle is equal to the search value.
We have found the element!
 - b) If the value in the middle is less than the search value.
Recursively search the second half of the container.
 - c) If the value in the middle is greater than the search value.
Recursively search the first half of the container.

value

	0	1	2	3	4	5	6	7
array	-6	-3	0	3	8	12	19	21

start

middle

end

value

-3

	0	1	2	3	4	5	6	7
array	-6	-3	0	3	8	12	19	21

start

0

middle

--

end

7

value

-3

	0	1	2	3	4	5	6	7
array	-6	-3	0	3	8	12	19	21

start

0

middle

end

7

$\text{middle} = \text{int}((\text{start} + \text{end}) / 2)$

value -3

	0	1	2	3	4	5	6	7
array	-6	-3	0	3	8	12	19	21

start 0

middle 3

end 7

$\text{middle} = \text{int}((\text{start} + \text{end}) / 2)$

value -3

	0	1	2	3	4	5	6	7
array	-6	-3	0	3	8	12	19	21

start 0

middle 1

end 2

$\text{middle} = \text{int}((\text{start} + \text{end}) / 2)$

value

-3

	0	1	2	3	4	5	6	7
array	-6	-3	0	3	8	12	19	21

start

0

middle

1

end

2

return element 1

value

19

	0	1	2	3	4	5	6	7
array	-6	-3	0	3	8	12	19	21

start

0

middle

--

end

7

value

19

	0	1	2	3	4	5	6	7
array	-6	-3	0	3	8	12	19	21

start

0

middle

3

end

7

$\text{middle} = \text{int}((\text{start} + \text{end}) / 2)$

value

19

	0	1	2	3	4	5	6	7
array	-6	-3	0	3	8	12	19	21

start

4

middle

--

end

7

$$\text{middle} = \text{int}((\text{start} + \text{end}) / 2)$$

value

19

	0	1	2	3	4	5	6	7
array	-6	-3	0	3	8	12	19	21

start

4

middle

5

end

7

$\text{middle} = \text{int}((\text{start} + \text{end}) / 2)$

value

19

	0	1	2	3	4	5	6	7
array	-6	-3	0	3	8	12	19	21

start

6

middle

6

end

7

$\text{middle} = \text{int}((\text{start} + \text{end}) / 2)$

value

19

	0	1	2	3	4	5	6	7
array	-6	-3	0	3	8	12	19	21

start

6

middle

6

end

7

return element 6

Iterative!

```
function BINARYSEARCH(ARRAY, VALUE)      (1)
  START = 0                               (2)
  END = size of ARRAY - 1                 (3)
  loop while START <= END                 (4)
    MIDDLE = INT((START + END) / 2) (5)
    if ARRAY[MIDDLE] == VALUE then (6)
      return MIDDLE                      (7)
    else if ARRAY[MIDDLE] > VALUE then (8)
      END = MIDDLE - 1                   (9)
    else if ARRAY[MIDDLE] < VALUE then (10)
      START = MIDDLE + 1                 (11)
    end if                               (12)
  end loop                               (13)
  return -1                              (14)
end function                              (15)
```


Iterative!

```
function BINARYSEARCH(ARRAY, VALUE)      (1)
  START = 0                               (2)
  END = size of ARRAY - 1                 (3)
  loop while START <= END                 (4)
    MIDDLE = INT((START + END) / 2) (5)
    if ARRAY[MIDDLE] == VALUE then (6)
      return MIDDLE                      (7)
    else if ARRAY[MIDDLE] > VALUE then (8)
      END = MIDDLE - 1                   (9)
    else if ARRAY[MIDDLE] < VALUE then (10)
      START = MIDDLE + 1                 (11)
    end if                               (12)
  end loop                               (13)
  return -1                              (14)
end function                             (15)
```

Iterative!

```
function BINARYSEARCH(ARRAY, VALUE)      (1)
  START = 0                               (2)
  END = size of ARRAY - 1                 (3)
  loop while START <= END                 (4)
    MIDDLE = INT((START + END) / 2) (5)
    if ARRAY[MIDDLE] == VALUE then (6)
      return MIDDLE                      (7)
    else if ARRAY[MIDDLE] > VALUE then (8)
      END = MIDDLE - 1                   (9)
    else if ARRAY[MIDDLE] < VALUE then (10)
      START = MIDDLE + 1                 (11)
    end if                               (12)
  end loop                               (13)
  return -1                              (14)
end function                             (15)
```

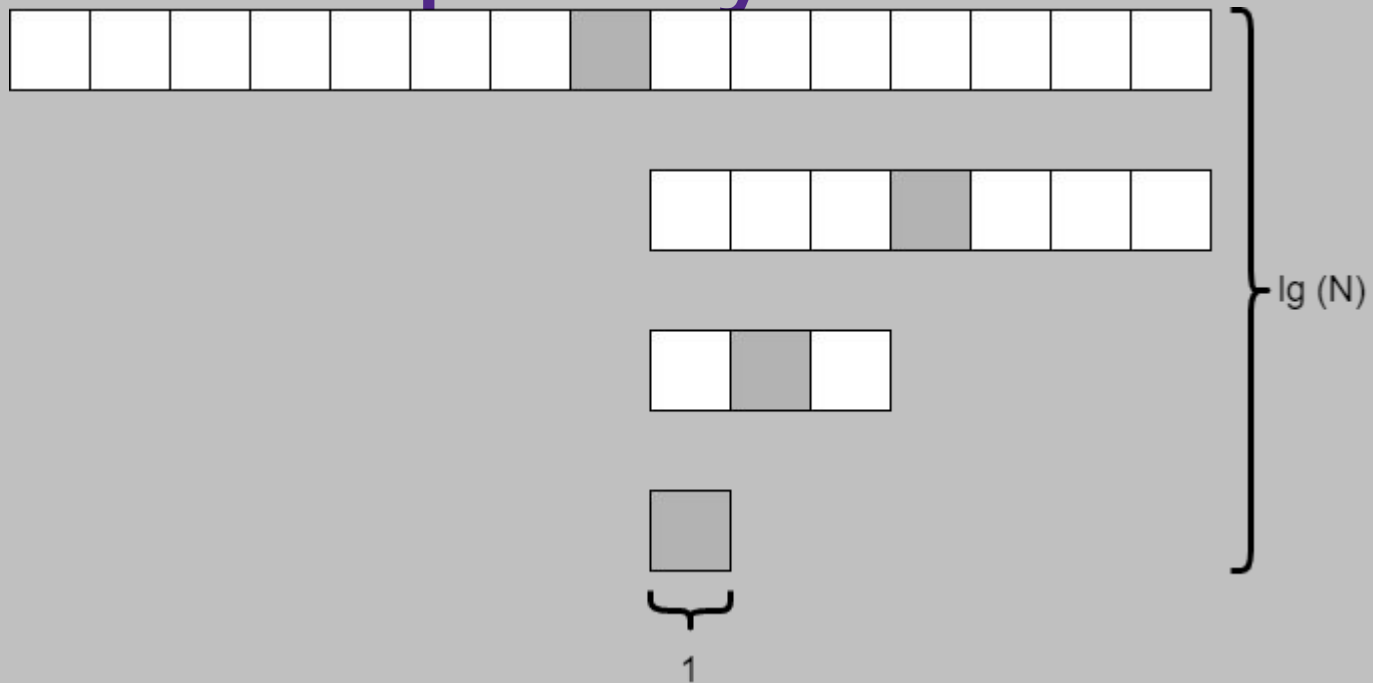
Iterative!

```
function BINARYSEARCH(ARRAY, VALUE)      (1)
  START = 0                               (2)
  END = size of ARRAY - 1                 (3)
  loop while START <= END                 (4)
    MIDDLE = INT((START + END) / 2) (5)
    if ARRAY[MIDDLE] == VALUE then (6)
      return MIDDLE                      (7)
    else if ARRAY[MIDDLE] > VALUE then (8)
      END = MIDDLE - 1                   (9)
    else if ARRAY[MIDDLE] < VALUE then (10)
      START = MIDDLE + 1                 (11)
    end if                               (12)
  end loop                               (13)
  return -1                              (14)
end function                             (15)
```

Recursive!

```
function BINARYSEARCHRECURSE(ARRAY, VALUE, START, END) (1)
  if START > END then (2)
    return -1 (3)
  end if (4)
  MIDDLE = INT((START + END) / 2) (5)
  if ARRAY[MIDDLE] == VALUE then (6)
    return MIDDLE (7)
  else if ARRAY[MIDDLE] > VALUE then (8)
    return BINARYSEARCHRECURSE(ARRAY, VALUE, START, MIDDLE - 1) (9)
  else if ARRAY[MIDDLE] < VALUE then (10)
    return BINARYSEARCHRECURSE(ARRAY, VALUE, MIDDLE + 1, END) (11)
  end if (12)
end function (13)
```

Time Complexity



Binary Search

0101010101010101
0101010101010101
0101010101010101
0101010101010101
0101010101010101



[This Photo](#) by Unknown Author
is licensed under [CC BY-SA](#)